INTERVIEW PREP • Q&A DRILLS • PYTHON EXAMPLES

Machine Learning Pocket Notes Interview

Q&A: 200+ Questions, Practical Python Examples, Chapter-by-Chapter Overview and Full Q&A Drill.

205 deep-dive interview questions across 17 chapters, plus 204 rapid-fire Q&A drills across 14 topics---from supervised learning fundamentals to Transformers, MLOps, and explainable AI.

aiengineeringinsider.com
aiengineeringinsider.substack.com/subscribe

LAMHOT SIAGIAN
AI ENGINEERING INSIDER

Contents

1 Foundations of Machine Learning

Chapter Overview

⋆ Key Insight

Machine learning is the discipline of designing algorithms that improve automatically through experience. Rather than hand-coding rules, we expose a model to data and let it discover patterns. The three canonical learning paradigms — supervised, unsupervised, and reinforcement learning — differ in what signal drives that discovery.

Supervised learning maps inputs $\mathbf{x}$ to outputs y using labelled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$. The learning algorithm minimises an empirical loss $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \ell(f_\theta(\mathbf{x}_i), y_i)$ and the hope is that the learnt f_θ generalises to unseen inputs. **Unsupervised learning** finds structure — clusters, latent factors, density estimates — without labels. **Reinforcement learning** trains an agent to maximise cumulative reward through interaction with an environment.

The **bias–variance trade-off** is the central tension in supervised learning: simpler models underfit (high bias), complex models overfit (high variance). Regularisation, cross-validation, and ensemble methods are the primary tools for navigating it. The **No Free Lunch theorem** formalises the intuition that no single learner is universally best: every algorithm's advantage on some problems is paid for by disadvantage on others.

Q1 — What is the difference between supervised, unsupervised, and reinforcement learning?

Q: Define each paradigm and give a concrete industry example of each.
Answer:

Supervised learning The model trains on labelled pairs $(\mathbf{x}, y)$ and learns a mapping $f : \mathbf{x} \to y$. The loss $\mathcal{L}$ measures prediction error and gradient descent adjusts parameters θ to minimise it. *Example:* fraud detection — each transaction is labelled *fraudulent* or *legitimate* and a gradient-boosted classifier learns the boundary.

Unsupervised learning No labels are provided. The algorithm discovers latent structure: clusters (k-means), manifolds (t-SNE/UMAP), generative distributions (VAEs, GANs), or low-rank factors (PCA, NMF). *Example:* customer segmentation — a retailer clusters purchase histories to identify distinct buyer personas without any pre-defined categories.

Reinforcement learning An agent observes state s_t, takes action a_t, and receives reward r_t from an environment. It learns a policy $\pi(a|s)$ that maximises expected cumulative discounted reward $\mathbb{E}[\sum_t \gamma^t r_t]$. *Example:* recommendation systems — a streaming platform trains an RL agent to select which video to surface next, optimising for long-term watch time rather than immediate clicks.

✓ Best Practice

In production, the boundaries blur: semi-supervised methods exploit cheap unlabelled data alongside scarce labels; self-supervised pre-training (BERT, GPT) uses unsupervised objectives on massive corpora then fine-tunes with supervision on small labelled sets. Always ask *"where does my signal come from?"* before selecting a paradigm.

Q2 — Explain the bias–variance trade-off. How does it guide model selection?

Q: Derive the bias–variance decomposition and explain its practical implications.
Answer:
For a regression target $y = f(\mathbf{x}) + \epsilon$ with $\mathbb{E}[\epsilon] = 0$, $\text{Var}[\epsilon] = \sigma^2$, the expected squared test error of estimator $\hat{f}$ at point $\mathbf{x}_0$ decomposes as:

$$\mathbb{E}\Big[(y - \hat{f}(\mathbf{x}_0))^2\Big] = \underbrace{\Big(f(\mathbf{x}_0) - \mathbb{E}[\hat{f}(\mathbf{x}_0)]\Big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}\Big[\hat{f}(\mathbf{x}_0)\Big]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

- **High bias** (underfitting): model is too simple to capture the true f. Error is high on *both* train and test sets. Fix: increase model capacity, add features, reduce regularisation.
- **High variance** (overfitting): model memorises training noise. Low train error, high test error. Fix: add regularisation (L1/L2, dropout), gather more data, reduce capacity, or use ensembles.
- **Sweet spot**: the right complexity minimises total generalisation error. Cross-validation on a held-out validation set estimates where this is without touching the test set.

! Pitfall / Warning

Deep neural networks appear to violate the classical trade-off in the *double descent* regime: as model size or training steps grow past interpolation threshold, test error can *decrease* again. This is an active research area; the classical bias–variance story still guides feature engineering and regularisation choices but should not be applied naively to very large networks.

Q3 — What is the curse of dimensionality and how do you address it?

Q: Explain why high-dimensional data is problematic and list mitigation strategies.
Answer:
In d dimensions, the volume of the unit hypercube grows as $1^d = 1$, but the fraction of data within distance ϵ of any point shrinks exponentially. Concretely, to cover 10% of the unit hypercube with a hypercube in $d = 10$ dimensions requires edge length $0.1^{1/10} \approx 0.79$ — almost the entire range. Consequences:

1. **Sparsity**: N training samples become increasingly sparse as d grows; the nearest neighbours of a query point are not meaningfully "near".
2. **Distance concentration**: the ratio max / min of pairwise distances converges to 1, making nearest-neighbour search uninformative.
3. **Exponential sample complexity**: parametric density estimation requires exponentially more data to achieve a fixed error level.

Mitigations:

- **Dimensionality reduction**: PCA (linear), autoencoders (non-linear), UMAP for visualisation. Retain 95% of explained variance as a heuristic threshold.
- **Feature selection**: Lasso (L1), mutual information, recursive feature elimination. Remove redundant or noisy features.

- **Regularisation**: penalise model complexity so it does not exploit noise in sparse regions.
- **Domain knowledge**: engineer low-dimensional representations that capture semantic meaning (e.g. word embeddings, graph node features).

2 Regression

Chapter Overview

Regression models predict a continuous target. **Linear regression** is the workhorse: it is analytically tractable, interpretable, and the basis for understanding more complex models. In practice most tabular regression problems use gradient-boosted trees (XGBoost, LightGBM) or neural networks, but linear models remain important baselines and diagnostic tools. Key concerns are feature scaling, multicollinearity, heteroscedasticity, and the choice of regularisation.

Q4 — Derive the ordinary least-squares estimator and state when it is BLUE.

Q: Walk through the OLS derivation and the Gauss–Markov conditions.
Answer:
Minimise the residual sum of squares $\text{RSS}(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2$:

$$\frac{\partial \,\text{RSS}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = \mathbf{0} \implies \hat{\boldsymbol{\beta}}_{\text{OLS}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

Under the **Gauss–Markov conditions** — (i) $\mathbb{E}[\boldsymbol{\epsilon}] = \mathbf{0}$, (ii) $\text{Cov}[\boldsymbol{\epsilon}] = \sigma^2\mathbf{I}$ (homoscedasticity and no autocorrelation), (iii) $\mathbf{X}$ has full column rank — $\hat{\boldsymbol{\beta}}_{\text{OLS}}$ is the **Best Linear Unbiased Estimator** (BLUE): among all linear unbiased estimators it has the smallest variance.

! Pitfall / Warning

$\mathbf{X}^\top\mathbf{X}$ is singular (non-invertible) when features are perfectly collinear or when $p > n$. Solutions: remove duplicate features, apply Ridge regression ($\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I}$), or use the Moore–Penrose pseudoinverse.

Q5 — Compare Ridge, Lasso, and Elastic Net regularisation. When do you use each?

Q: Explain the penalties, their geometric intuition, and practical selection criteria.
Answer:
All three add a penalty to the OLS objective to shrink coefficients and reduce variance:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \underbrace{\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2}_{\text{fit}} + \lambda \underbrace{\Omega(\boldsymbol{\beta})}_{\text{penalty}}$$

Ridge (ℓ_2): $\Omega = \|\boldsymbol{\beta}\|_2^2$. Closed-form solution $(\mathbf{X}^\top\mathbf{X}+\lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$. Shrinks all coefficients smoothly; never sets them exactly to zero. Best when many features each contribute a small amount (dense signal). Handles multicollinearity by stabilising $\mathbf{X}^\top\mathbf{X}$.

Lasso (ℓ_1): $\Omega = \|\boldsymbol{\beta}\|_1$. No closed form; solved with coordinate descent or ISTA/ADMM. Produces sparse solutions (many coefficients exactly zero) due to the non-differentiability at the origin. Best

for automatic feature selection when you suspect only a few features matter.

Elastic Net: $\Omega = \alpha\|\boldsymbol{\beta}\|_1 + (1-\alpha)\|\boldsymbol{\beta}\|_2^2$. Combines both penalties. Selects sparse groups of correlated features (Lasso alone arbitrarily picks one from a correlated set). Use when $p \gg n$ and features are correlated.

Hyperparameter selection: always use cross-validation over a log-spaced grid of λ. Standardise features first — ℓ_1/ℓ_2 penalties are not scale-invariant.

Ridge vs Lasso -- Python Example

```python
import numpy as np
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Synthetic dataset: 200 samples, 50 features, only 10 informative
X, y = make_regression(n_samples=200, n_features=50,
                       n_informative=10, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

alphas = np.logspace(-3, 3, 50)

ridge = RidgeCV(alphas=alphas, cv=5).fit(X_train, y_train)
lasso = LassoCV(alphas=alphas, cv=5, max_iter=10_000).fit(X_train, y_train)
enet  = ElasticNetCV(l1_ratio=[.1,.5,.9,.99], alphas=alphas,
                     cv=5, max_iter=10_000).fit(X_train, y_train)

for name, model in [("Ridge", ridge), ("Lasso", lasso), ("ElasticNet", enet)]:
    r2   = model.score(X_test, y_test)
    nnz  = np.sum(model.coef_ != 0)
    print(f"{name:12s}  R2={r2:.4f}  non-zero coefs={nnz}/50")
```

3 Classification

Chapter Overview

Classification assigns discrete labels to inputs. The choice of algorithm depends on data size, feature types, interpretability requirements, and the cost of different error types. Logistic regression provides probabilistic outputs and is highly interpretable; tree-based ensembles (Random Forest, XGBoost) are powerful on tabular data; neural networks excel on unstructured data. Evaluating classifiers requires going beyond accuracy: precision, recall, F1, AUC-ROC, and calibration curves each illuminate different failure modes.

Q6 — Explain logistic regression: the model, training objective, and its probabilistic interpretation.

Q: Derive the logistic regression training objective from maximum likelihood.
Answer:
Logistic regression models the posterior probability of class 1 given input $\mathbf{x}$:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}}}$$

The log-likelihood over N i.i.d. samples is:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Maximising ℓ is equivalent to minimising the **binary cross-entropy loss**. Unlike OLS, there is no closed-form solution; optimisation uses gradient descent or Newton–Raphson (L-BFGS), both of which converge to the global optimum because the objective is convex.

✓ Best Practice

Calibration matters. The sigmoid output is only well-calibrated when the training distribution matches the deployment distribution and the model is not over- or under-regularised. Use Platt scaling or isotonic regression to calibrate any classifier before using its score as a probability in a downstream decision system.

Q7 — When would you choose Random Forest over XGBoost, or vice versa?

Q: Compare the two ensemble methods on training time, hyperparameters, and use-case fit.
Answer:
Both are tree ensembles that dominate tabular ML benchmarks, but they differ in mechanism:

Random Forest (bagging + feature randomness): Trains many deep trees independently in parallel, each on a bootstrap sample and a random feature subset. Predictions are averaged (regression) or majority-voted (classification). Robust to noisy features. Tolerant of default hyperparameters. Scales trivially across CPU cores. *Choose when*: you need a strong, interpretable baseline quickly; training data has many redundant or noisy features; hardware is limited.

XGBoost / LightGBM / CatBoost (gradient boosting): Trains shallow trees sequentially, each fitting the negative gradient of the loss from the previous ensemble. The sequential nature gives higher accuracy on most Kaggle-style datasets but makes it more sensitive to hyperparameters (learning rate, tree depth, subsampling ratios) and harder to parallelise. LightGBM's leaf-wise growth and histogram binning make it the fastest on large datasets. *Choose when*: raw predictive performance is paramount; you have the budget to tune hyperparameters; dataset is large and tabular.

From the Field

In production ML pipelines for structured data, my default is: prototype with Random Forest for a quick, robust baseline; switch to LightGBM with Optuna-tuned hyperparameters when squeezing out the last few percent of AUC matters for business metrics.

Q8 — Explain precision, recall, F1, and AUC-ROC. How do class imbalance and decision threshold affect them?

Q: Define each metric and state when you would prioritise one over another.
Answer:
Given confusion matrix entries TP, FP, TN, FN:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

AUC-ROC plots TPR vs FPR at every classification threshold and measures the probability that the model ranks a random positive above a random negative. It is threshold-independent and insensitive to class imbalance — making it the preferred reporting metric for imbalanced problems.
Threshold decisions: lowering the classification threshold increases recall (catch more positives) at the cost of precision (more false alarms). The right threshold depends on the cost asymmetry: in cancer screening, a missed diagnosis (false negative) is far more costly than a false alarm, so you tune for high recall. In spam filtering, user experience degrades with false positives, so you tune for high precision.

! Pitfall / Warning

Accuracy is misleading under class imbalance. A classifier that always predicts the majority class achieves 99% accuracy on a dataset with 1% positives — and zero recall. Always report precision/recall/F1 or AUC for imbalanced problems. Consider also **AUC-PR** (precision–recall curve area), which is more informative than AUC-ROC when positives are rare.

4 Decision Trees and Ensemble Methods

Chapter Overview

Decision trees partition the feature space with axis-aligned splits. A single tree is interpretable but has high variance — small changes in data can produce very different trees. Ensemble methods — bagging, boosting, and stacking — reduce variance or bias by combining many trees. Understanding the splitting criteria (Gini impurity, entropy, MSE) and stopping conditions (depth, min-leaf-samples) is essential for both trees and forests.

Q9 — How does a decision tree choose which feature and split point to use?

Q: Explain the splitting criteria for classification (Gini/entropy) and regression (MSE), and discuss tree pruning.
Answer:
At each node, the tree evaluates all $(d \times B)$ candidate (feature, threshold) pairs and selects the one that maximises the **information gain** (reduction in impurity/error):

$$\Delta I = I(\text{parent}) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R)$$

Impurity measures:

- **Gini impurity:** $I(t) = 1 - \sum_k p_k^2$. Ranges 0 (pure) to $1 - 1/K$ (uniform). Computationally cheaper than entropy; standard in sklearn.

- **Entropy / Information gain:** $I(t) = -\sum_k p_k \log_2 p_k$. Slightly favours balanced splits. Preferred in ID3/C4.5.
- **MSE (regression):** $I(t) = \frac{1}{n}\sum_i (y_i - \bar{y})^2$. Minimising variance within leaf regions.

Pruning strategies:

Pre-pruning (early stopping): Limit tree depth, require minimum samples per split/leaf, or only split if gain $> \epsilon$.

Post-pruning (cost-complexity): Grow full tree, then prune back by penalising complexity: $R_\alpha(T) = R(T) + \alpha|T|$ where $|T|$ is the number of leaves. Tune α with cross-validation.

Q10 — Explain gradient boosting from first principles. What problem does it solve that AdaBoost does not?

Q: Derive the gradient boosting update and compare to AdaBoost.
Answer:
Gradient Boosting (Friedman 2001) frames boosting as **functional gradient descent**: at step m, fit a tree h_m to the negative gradient of the loss $\mathcal{L}$ with respect to the current prediction $F_{m-1}(\mathbf{x})$:

$$r_i^{(m)} = -\left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$$

Then update: $F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \cdot h_m(\mathbf{x})$, where ν is the learning rate (shrinkage).
Key insight: because the target for tree m is the negative gradient of *any* differentiable loss, gradient boosting handles regression, classification, ranking, and custom losses (quantile, Huber) with the same algorithm. AdaBoost is equivalent to gradient boosting with exponential loss, which makes it sensitive to outliers — gradient boosting with Huber or log loss is far more robust.

✓ Best Practice

Modern implementations (XGBoost, LightGBM) add second-order Taylor expansion of the loss (Newton boosting), column/row subsampling, and histogram binning. These additions give 10–100× speedups over naive gradient boosting with better accuracy. Always start with LightGBM for tabular data.

5 Support Vector Machines

Chapter Overview

Support Vector Machines (SVMs) find the maximum-margin hyperplane that separates classes. The **kernel trick** allows SVMs to operate implicitly in infinite-dimensional feature spaces without computing the feature map explicitly. While neural networks have displaced SVMs in most deep learning applications, SVMs remain competitive on small, high-dimensional datasets (text, genomics) and provide a clean mathematical framework for understanding margin-based learning.

Q11 — Derive the SVM primal and dual. What is the kernel trick?

Q: Walk through the hard-margin SVM optimisation, its Lagrangian dual, and explain kernels.
Answer:
Primal (hard margin): find $\mathbf{w}, b$ maximising the geometric margin $2/\|\mathbf{w}\|$, equivalently:

$$\min_{\mathbf{w},b} \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \; \forall i$$

Introducing Lagrange multipliers $\alpha_i \geq 0$ and applying KKT stationarity conditions gives the **dual**:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \alpha_i \geq 0, \; \sum_i \alpha_i y_i = 0$$

The dual only depends on inputs through inner products $\mathbf{x}_i^\top \mathbf{x}_j$. **The kernel trick** replaces this with $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ for some (possibly infinite-dimensional) feature map ϕ, without ever computing ϕ explicitly. Popular kernels: RBF $e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$, polynomial $(1 + \mathbf{x}^\top \mathbf{x}')^d$, string kernels for text.
Soft-margin SVM introduces slack variables $\xi_i \geq 0$ and penalty $C\sum_i \xi_i$, trading margin width for training error tolerance. C is the primary regularisation hyperparameter.

6 Neural Networks and Deep Learning

Chapter Overview

Neural networks are universal function approximators composed of parameterised linear transformations followed by non-linear activations. Their power lies in **representation learning**: intermediate layers automatically construct hierarchical features from raw inputs. The modern deep learning stack — backpropagation, batch normalisation, residual connections, attention, and large-scale pretraining — has enabled breakthrough performance across vision, language, speech, and scientific domains.

Q12 — Explain backpropagation. Derive gradients for a two-layer network.

Q: Describe forward pass, loss computation, and gradient flow. Why does vanishing gradient occur?
Answer:
Backpropagation is the chain rule applied recursively to compute $\partial\mathcal{L}/\partial\theta_l$ for every parameter layer l. For a two-layer network with inputs $\mathbf{x}$, hidden layer $\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$, and output $\hat{y} = \mathbf{W}_2\mathbf{h} + b_2$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \delta_2 \mathbf{h}^\top, \qquad \delta_2 = \frac{\partial \mathcal{L}}{\partial \hat{y}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \delta_1 \mathbf{x}^\top, \qquad \delta_1 = \mathbf{W}_2^\top \delta_2 \odot \sigma'(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

Vanishing gradients: when σ' is small (e.g. sigmoid saturates at 0 or 1), δ_1 becomes exponentially small after many layers, and early layers stop learning. Solutions: ReLU activation (gradient $= 1$ for positive pre-activations), batch normalisation, residual connections (skip $\partial\mathcal{L}/\partial\mathbf{x}$ directly through identity path), and careful weight initialisation (He, Xavier).

```
Manual backprop for 2-layer network
```

```python
import numpy as np

def relu(z):       return np.maximum(0, z)
def relu_grad(z):  return (z > 0).astype(float)

def forward(X, W1, b1, W2, b2):
    z1 = X @ W1.T + b1
    h  = relu(z1)
    z2 = h @ W2.T + b2
    return z1, h, z2

def backward(X, y, z1, h, z2, W1, W2):
    N = X.shape[0]
    # MSE loss gradient w.r.t. output
    dL_dz2 = 2 * (z2.squeeze() - y) / N   # (N,)
    dL_dW2 = dL_dz2[:, None].T @ h         # (out, hidden)
    dL_db2 = dL_dz2.sum()
    # Backprop through W2 and ReLU
    dL_dh  = dL_dz2[:, None] @ W2          # (N, hidden)
    dL_dz1 = dL_dh * relu_grad(z1)         # (N, hidden)
    dL_dW1 = dL_dz1.T @ X                  # (hidden, in)
    dL_db1 = dL_dz1.sum(axis=0)
    return dL_dW1, dL_db1, dL_dW2, dL_db2
```

Q13 — Compare Adam, SGD with momentum, and RMSProp. When should you prefer each?

Q: Explain the update rules and the practical trade-offs in convergence speed and generalisation.
Answer:

SGD + Momentum $\mathbf{v} \leftarrow \beta\mathbf{v} - \eta\nabla\mathcal{L}$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$. Accumulates gradient direction; escapes shallow local minima. Sensitive to learning rate choice. Often finds flatter minima than adaptive methods, which empirically generalise better on held-out data. *Default for large-scale vision tasks (ImageNet training).*

RMSProp $\mathbf{v} \leftarrow \beta\mathbf{v} + (1-\beta)(\nabla\mathcal{L})^2$; $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{\sqrt{\mathbf{v}+\epsilon}}\nabla\mathcal{L}$. Adapts learning rate per-parameter by dividing by a running RMS of gradients. Excels on non-stationary objectives (RNNs).

Adam Combines momentum (first moment $\mathbf{m}$) with RMSProp (second moment $\mathbf{v}$), with bias correction. Effective across many architectures with minimal tuning. However, it can converge to sharper minima that generalise less well than SGD. *Default choice for Transformers and most NLP tasks.*

✓ Best Practice

Use Adam with a warmup + cosine annealing schedule for Transformers. Switch to SGD with momentum and a step-decay schedule for ResNets if you need maximum test accuracy. Tune learning rate first (most impactful hyperparameter), then weight decay.

7 Convolutional Neural Networks

Chapter Overview

Convolutional Neural Networks (CNNs) exploit the spatial structure of images through parameter sharing and local connectivity. A convolution operation slides a small filter across the input, computing dot products at each position and producing a feature map. Stacking convolutions, pooling, batch normalisation, and non-linearities builds up a hierarchy of features: edges and textures in early layers, object parts and semantics in later layers. CNNs revolutionised computer vision and remain foundational even as Vision Transformers (ViTs) increasingly challenge them at scale.

Q14 — How does a convolution operation work, and why is parameter sharing important?

Q: Describe the convolution computation, receptive field, and the benefit of weight sharing.
Answer:
A 2D convolution with kernel $\mathbf{K} \in \mathbb{R}^{k \times k}$ applied to input feature map $\mathbf{X} \in \mathbb{R}^{H \times W}$ produces output:

$$\mathbf{Y}[i,j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \mathbf{X}[i+u, j+v] \cdot \mathbf{K}[u,v]$$

Output spatial dimensions: $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$ (padding p, stride s).
Parameter sharing: the same k^2 kernel weights are used at every spatial location. This gives two benefits: (1) *drastically fewer parameters* vs. a fully connected layer (e.g. a 3×3 filter applied to a 256×256 image uses 9 weights instead of $256^2 \approx 65{,}000$); (2) *translation equivariance* — a shifted input produces a shifted output, which is exactly the right inductive bias for image recognition.
Receptive field of a unit in layer l is the input region that influences it. With kernel size k and stride 1, receptive field grows as $(l(k-1)+1)^2$. Dilated convolutions and pooling layers grow it faster without adding parameters.

8 Recurrent Networks and Sequence Models

Chapter Overview

Recurrent Neural Networks (RNNs) process sequences by maintaining a hidden state $\mathbf{h}_t$ that summarises history up to step t. Vanilla RNNs suffer from vanishing gradients over long sequences. LSTMs and GRUs introduce gating mechanisms that allow gradients to flow over hundreds of steps. Since 2017, Transformer-based models have displaced RNNs in most sequence tasks by processing the entire sequence in parallel through self-attention. Understanding RNNs remains important for time-series tasks and edge deployment where compute is constrained.

Q15 — Why do vanilla RNNs fail on long sequences, and how do LSTMs fix this?

Q: Describe the vanishing gradient problem in RNNs and the LSTM gating mechanism.
Answer:
The RNN update is $\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t)$. Backpropagating through T steps multiplies Jacobians:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_1} = \prod_{t=2}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \prod_{t=2}^{T} \text{diag}[\tanh'(\cdot)] \mathbf{W}_h$$

When $\|\mathbf{W}_h\| < 1$ or $\tanh' < 1$, this product shrinks exponentially — **vanishing gradients**. When $\|\mathbf{W}_h\| > 1$ it can explode (handled by gradient clipping).
LSTM (Hochreiter & Schmidhuber, 1997) introduces a **cell state** $\mathbf{c}_t$ that runs in parallel with additive (not multiplicative) updates, allowing gradients to flow:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_f) && \text{(forget gate)}\\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_i) && \text{(input gate)}\\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_c) && \text{(candidate cell)}\\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{(cell update)}\\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_o) && \text{(output gate)}\\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{(hidden state)}
\end{aligned}$$

The key insight: gradient flows through $\mathbf{c}_t \leftarrow \mathbf{c}_{t-1}$ via multiplication by $\mathbf{f}_t \in (0,1)$, which can stay close to 1 for relevant memory, preventing collapse over hundreds of steps.

9 Transformers and Attention

Chapter Overview

The Transformer (Vaswani et al., 2017) replaced recurrence with **self-attention**: each token attends to all other tokens simultaneously, enabling parallelism during training and capturing long-range dependencies without gradient pathology. Combined with positional encodings, residual connections, and layer normalisation, Transformers scale predictably with data and compute — the foundation of GPT, BERT, T5, and every large language model in production today.

Q16 — Derive scaled dot-product attention. Why the scaling factor $\sqrt{d_k}$?

Q: Write out the attention formula and explain multi-head attention.
Answer:
Given queries $\mathbf{Q} \in \mathbb{R}^{n\times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{m\times d_k}$, values $\mathbf{V} \in \mathbb{R}^{m\times d_v}$:

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

Why $\sqrt{d_k}$? The dot product $\mathbf{q}^\top\mathbf{k}$ has variance d_k when $\mathbf{q},\mathbf{k}$ are unit-variance random vectors. Without scaling, the softmax receives inputs with large magnitude, producing near-one-hot attention distributions and near-zero gradients (softmax saturation). Dividing by $\sqrt{d_k}$ restores unit variance and ensures informative gradients throughout training.
Multi-head attention runs H parallel attention heads with separate learned projections $\mathbf{W}_i^Q, \mathbf{W}_i^K, \mathbf{W}_i^V$ then concatenates:

$$\text{MHA}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Concat}(\text{head}_1,\ldots,\text{head}_H)\mathbf{W}^O$$

Each head can attend to different aspects of the input (syntax, coreference, semantics), and the concatenation enriches the representation at no extra depth.

✓ Best Practice

Flash Attention (Dao et al., 2022) reimplements attention with IO-aware tiling to avoid materialising the full $n \times n$ attention matrix in HBM, reducing memory from $O(n^2)$ to $O(n)$ and achieving 2–4× speedups. Always use Flash Attention for sequences longer than ∼512 tokens in production.

10 Unsupervised Learning and Dimensionality Reduction

Chapter Overview

Unsupervised methods extract structure from unlabelled data. **Clustering** (k-means, DBSCAN, hierarchical) groups similar instances. **Dimensionality reduction** (PCA, t-SNE, UMAP, autoencoders) projects data to lower-dimensional manifolds for visualisation, compression, and feature extraction. **Density estimation** (GMMs, normalising flows, VAEs) models the data distribution for generation, anomaly detection, and imputation. These techniques are increasingly important as labelled data remains scarce and self-supervised pretraining on unlabelled data dominates frontier ML.

Q17 — Explain the k-means algorithm. What are its limitations and how does k-means++ address initialisation?

Q: Describe the Lloyd's algorithm update steps, convergence guarantees, and practical failure modes.
Answer:
Lloyd's algorithm (k-means):

1. Initialise K centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$.
2. **Assignment step**: assign each point to its nearest centroid: $c_i = \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$.
3. **Update step**: recompute centroids as cluster means: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$.
4. Repeat until assignments do not change.

The objective $J = \sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|^2$ (within-cluster sum of squares) decreases monotonically and is bounded below, so convergence is guaranteed — to a *local* minimum.
Limitations: assumes spherical clusters of equal size; sensitive to outliers (squared distance penalises them heavily); requires pre-specifying K; deterministic results depend on initialisation.
k-means++ initialisation: choose first centroid uniformly at random; each subsequent centroid is chosen with probability proportional to its squared distance from the nearest existing centroid. This spreads centroids across the data and provides an $O(\log K)$ approximation guarantee to the optimal objective.

11 Feature Engineering and Preprocessing

Chapter Overview

In industrial ML systems, feature engineering — transforming raw data into informative representations — often determines model quality more than algorithm choice. This chapter covers scaling, encoding, handling missing data, and creating informative features. A well-engineered feature set on a simple model regularly outperforms a poorly prepared feature set on a complex one.

Q18 — How do you handle missing data? Compare deletion, imputation, and model-based approaches.

Q: Classify missing data mechanisms and describe appropriate handling strategies.
Answer:
Missing data mechanisms (Rubin, 1976):

MCAR (Missing Completely At Random): Missingness is independent of observed and missing values. Complete-case analysis (listwise deletion) is unbiased but reduces sample size.

MAR (Missing At Random): Missingness depends only on observed values. Imputation conditional on observed values is valid.

MNAR (Missing Not At Random): Missingness depends on the missing value itself (e.g. high earners omit salary). Requires domain-specific handling; imputation alone introduces bias.

Strategies:

- **Deletion**: safe only under MCAR and when missingness rate is low ($< 5\%$).
- **Simple imputation**: mean/median (continuous), mode or "Unknown" category (categorical). Fast; can distort distributions and underestimate variance.
- **Multiple imputation** (MICE): create M complete datasets with different imputed values, train model on each, pool results. Correctly propagates uncertainty; computationally expensive.
- **Model-based**: train a regressor/classifier to predict missing values from observed ones. KNN imputation is effective when data has local structure.
- **Indicator variable**: add a binary "was_missing" feature alongside imputed value. Lets the model learn from the missingness pattern itself — especially useful under MNAR.

! Pitfall / Warning

Always fit your imputer on the training set and *transform* the validation/test sets using training statistics. Fitting on the full dataset causes data leakage and will give optimistic evaluation metrics.

12 Model Evaluation and Selection

Chapter Overview

Rigorous evaluation is what separates a research experiment from a production system. This chapter covers cross-validation strategies, the correct use of train/val/test splits, hyperparameter optimisation, and the statistical tests needed to determine whether performance differences are real or random.

Q19 — Explain k-fold cross-validation. When would you use stratified, grouped, or time-series CV?

Q: Describe the CV procedure and state when each variant is appropriate.
Answer:
Standard k-fold CV: randomly partition data into k equal folds. Train on $k-1$ folds, evaluate on the held-out fold, rotate, and average the k scores. Typical $k = 5$ or 10 balances bias (larger $k \to$ lower bias in estimate) and variance (larger $k \to$ higher variance, more compute).
When to use variants:

Stratified k-fold: Maintains class proportions in each fold. Essential for imbalanced classification (e.g. 1% fraud rate); random splits could put zero positives in a fold.

Group k-fold: Ensures all records from a given entity (patient, user, store) appear in only one fold. Use when records within a group are correlated (repeated measurements). Prevents data leakage from the same person appearing in train and test.

Time-series / walk-forward CV: Train on $t_1, \ldots, t_k$, validate on $t_{k+1}, \ldots, t_{k+h}$, advance window. Respects temporal ordering; prevents future data leaking into training. Use for any time-series forecasting or temporal prediction task.

13 Bayesian Methods

Chapter Overview

Bayesian methods frame learning as probabilistic inference: we maintain a prior over model parameters, update it with observed data to obtain a posterior, and integrate over the posterior to make predictions. This principled uncertainty quantification is invaluable in high-stakes applications (medicine, autonomous driving, finance) where knowing *what the model does not know* is as important as its point predictions.

Q20 — Explain Bayesian inference. What is the trade-off between Bayesian and frequentist approaches?

Q: State Bayes' theorem in the context of ML and discuss practical inference challenges.
Answer:
Given data $\mathcal{D}$ and parameters $\boldsymbol{\theta}$, Bayes' theorem gives:

$$\underbrace{p(\boldsymbol{\theta} \mid \mathcal{D})}_{\text{posterior}} = \frac{\underbrace{p(\mathcal{D} \mid \boldsymbol{\theta})}_{\text{likelihood}} \cdot \underbrace{p(\boldsymbol{\theta})}_{\text{prior}}}{\underbrace{p(\mathcal{D})}_{\text{marginal likelihood}}}$$

Prediction integrates over the posterior: $p(y^* \mid \mathbf{x}^*, \mathcal{D}) = \int p(y^* \mid \mathbf{x}^*, \boldsymbol{\theta}) p(\boldsymbol{\theta} \mid \mathcal{D})\, d\boldsymbol{\theta}$.
Advantages of Bayesian approach: principled uncertainty quantification; automatically regularises via the prior; small-data regime performance; can incorporate domain knowledge in the prior; naturally handles model comparison via marginal likelihood.
Challenges: the marginal likelihood $p(\mathcal{D}) = \int p(\mathcal{D} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})\, d\boldsymbol{\theta}$ is intractable for most models. Practical solutions: Laplace approximation, variational inference (ELBO), Markov Chain Monte Carlo (MCMC), or approximate methods like Bayesian neural networks with MC Dropout.
Frequentist methods (MLE, OLS, hypothesis testing) are simpler, computationally cheap, and work well with large data. Bayesian methods shine when data is scarce, uncertainty is critical, or sequential

updating is needed (online learning, A/B tests with early stopping).

14 Natural Language Processing

Chapter Overview

NLP has undergone two revolutions: the first was word embeddings (Word2Vec, GloVe) that gave words continuous vector representations; the second was the Transformer, which enabled models trained on internet-scale corpora to be fine-tuned with a handful of labelled examples for almost any task. Modern NLP is dominated by large language models (LLMs): BERT for understanding, GPT for generation, T5 for seq2seq, and their successors.

Q21 — Explain word embeddings: Word2Vec, GloVe, and contextual embeddings (BERT). What are their key differences?

Q: Describe how each representation is trained and when you would use each.
Answer:

Word2Vec (Mikolov et al., 2013): Static embeddings trained with a shallow neural network on local context windows. Two objectives: CBOW (predict centre word from context) and Skip-gram (predict context from centre word). Captures syntactic and semantic regularities: `king - man + woman` $\approx$ `queen`. *Limitation*: a word has one vector regardless of context ("bank" as riverbank vs. financial institution).

GloVe (Pennington et al., 2014): Factorises the global word co-occurrence matrix. Combines global statistics (like LSA) with the local context structure of Word2Vec. Often slightly better on analogy and similarity benchmarks.

BERT contextual embeddings: A Transformer encoder pre-trained with Masked Language Modelling (MLM) and Next Sentence Prediction (NSP) on large corpora. Each token receives a *context-dependent* representation: "bank" in two different sentences produces two different vectors. Fine-tuning on downstream tasks achieves state-of-the-art across NLU benchmarks.

When to use: Word2Vec/GloVe are lightweight feature extractors for simple text classifiers (TF-IDF + LogReg) or when compute is constrained. BERT-family models are the default for any task requiring deep language understanding.

15 Reinforcement Learning

Chapter Overview

Reinforcement learning (RL) trains agents to maximise cumulative reward through trial and error in an environment. The Markov Decision Process (MDP) framework formalises the setting: states, actions, transition dynamics, and a reward function. Key algorithms divide into value-based methods (Q-learning, DQN), policy gradient methods (REINFORCE, PPO, SAC), and model-based RL. RL powers game-playing agents (AlphaGo, Dota2), robotics, recommender systems, and RLHF for aligning LLMs.

Q22 — Explain the Bellman equation and Q-learning. Why is experience replay and a target network needed in DQN?

Q: Derive the Bellman optimality equation and describe the DQN stabilisation tricks.
Answer:
The **Bellman optimality equation** defines the optimal action-value function $Q^*(s, a)$:

$$Q^*(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \;\middle|\; s, a\right]$$

Q-learning tabularly iterates: $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$. This is off-policy (behaviour policy can differ from greedy policy) and model-free (no transition dynamics needed).
DQN (Mnih et al., 2015) approximates $Q(s, a; \boldsymbol{\theta})$ with a neural network. Two instability sources arise and two tricks fix them:

Experience replay: Store (s, a, r, s') transitions in a replay buffer; sample mini-batches i.i.d. during training. Breaks temporal correlations that would make gradient estimates highly correlated and biased.

Target network: Maintain a frozen copy $\boldsymbol{\theta}^-$ of the network updated every C steps. Compute TD targets as $r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta}^-)$ instead of using $\boldsymbol{\theta}$ itself. Prevents the bootstrap target from "chasing its own tail" and causing divergence.

16 MLOps and Production ML

Chapter Overview

MLOps bridges model development and reliable production deployment. A model that achieves 95% accuracy in a notebook is worthless if it cannot be served reliably, monitored for drift, retrained automatically, and explained to stakeholders. This chapter covers the deployment lifecycle: model packaging (Docker, ONNX), serving (REST APIs, batch inference), monitoring (data drift, concept drift, performance degradation), and CI/CD pipelines for ML.

Q23 — What is data drift and concept drift? How do you detect and respond to each?

Q: Define both drift types, describe detection methods, and outline a monitoring architecture.
Answer:

Data drift (covariate shift): The distribution of input features $P(\mathbf{x})$ changes between training and serving, while the conditional $P(y|\mathbf{x})$ remains stable. Example: a fraud model trained before a pandemic sees very different transaction patterns post-pandemic.

Concept drift: The relationship $P(y|\mathbf{x})$ itself changes. Example: customer preferences shift with market trends; yesterday's high-quality lead becomes today's disinterested prospect.

Detection methods:

- **Statistical tests on input features**: Population Stability Index (PSI), Kolmogorov–Smirnov test, Maximum Mean Discrepancy (MMD) between training distribution and recent serving data.

- **Performance monitoring**: track accuracy, precision, recall, AUC on labelled production data (where ground truth is available with delay). Alert when metrics drop below a threshold.
- **Proxy metrics**: if labels arrive with delay, monitor prediction distribution shift as an early warning.

Response:

- **Retraining schedule**: periodic (weekly/monthly) or triggered by drift alerts.
- **Incremental learning**: update model weights on a sliding window of recent data.
- **Ensemble with recency bias**: blend old and freshly-trained models, upweighting the newer one as confidence grows.

✓ Best Practice

Build a **drift monitoring pipeline** as a first-class citizen alongside your serving infrastructure. Log every prediction with its input features and prediction timestamp. Use a tool like Evidently AI, WhyLogs, or a custom pipeline on your data warehouse to compute drift metrics daily and push alerts to Slack/PagerDuty.

Q24 — Describe a production ML serving architecture. How do you handle latency, throughput, and reliability?

Q: Design a low-latency ML API service and discuss the engineering trade-offs.
Answer:
A robust serving architecture separates concerns across four layers:

1. **Model registry**: versioned model artefacts stored in S3/GCS with metadata (training date, data version, performance metrics). Tools: MLflow, Weights & Biases Registry, SageMaker Model Registry.
2. **Serving layer**: containerised inference server (TorchServe, Triton Inference Server, or a custom FastAPI wrapper) behind a load balancer. Export models to ONNX or TensorRT for hardware-optimised inference. Batch inference requests (dynamic batching) to amortise fixed GPU overhead.
3. **Feature store**: pre-computed, low-latency feature serving (Redis, Feast) for features that are expensive to compute at request time. Ensures training-serving consistency — the same feature computation code is used in both pipelines.
4. **Monitoring and logging**: capture input features, prediction scores, and ground-truth labels (when available) to an event stream (Kafka) for drift detection, debugging, and offline analysis.

Latency trade-offs:

- Use **model quantisation** (INT8/FP16) and **pruning** to reduce model size and inference time.
- Cache predictions for high-frequency, low-cardinality inputs (e.g. item popularity scores).
- Set **P99 latency SLOs** (e.g. $<$50ms), not just average latency. Tail latency determines user experience.

17 Explainability and Ethics in ML

Chapter Overview

As ML systems make consequential decisions — credit, hiring, medical diagnosis, criminal justice — the ability to explain *why* a model made a prediction is both a regulatory requirement (GDPR Article 22, EU AI Act) and a scientific necessity. This chapter covers model-agnostic explanation methods (SHAP, LIME), intrinsically interpretable models, and the ethical considerations around fairness, accountability, and transparency.

Q25 — Explain SHAP values. Why are they theoretically grounded compared to other feature importance methods?

Q: Define SHAP, derive the Shapley value axioms, and compare to permutation importance and LIME.
Answer:
SHAP (SHapley Additive exPlanations, Lundberg & Lee 2017) assigns each feature j a value ϕ_j representing its contribution to the prediction $f(\mathbf{x})$ relative to a baseline $\mathbb{E}[f(\mathbf{X})]$:

$$f(\mathbf{x}) = \mathbb{E}[f(\mathbf{X})] + \sum_{j=1}^{p} \phi_j$$

SHAP values are the unique attribution satisfying four axioms from cooperative game theory:

- **Efficiency**: $\sum_j \phi_j = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{X})]$ (attributions sum to prediction).
- **Symmetry**: equally contributing features get equal attributions.
- **Dummy**: a feature that never changes the prediction gets attribution 0.
- **Linearity**: attribution over a mixture of models equals the mixture of attributions.

The Shapley value for feature j averages the marginal contribution of j over all possible feature subsets S:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} \left[f_{\mathbf{x}}(S \cup \{j\}) - f_{\mathbf{x}}(S) \right]$$

Comparison:

Permutation importance: Measures drop in accuracy when feature is shuffled. Fast; global; can underestimate importance of correlated features.

LIME: Fits a local linear model around the instance. Simple to explain; unstable (results vary with sampling); not additive.

SHAP: Theoretically grounded; consistent; local and global; computationally expensive for exact computation but TreeSHAP is $O(TLD^2)$ for tree ensembles.